NEXORA PHASE 4A — TECHNICAL SPECIFICATION

Core Storage & Multi-Format Ingestion Engine

Version: 1.0.0 | Date: 2026-06-26

Priority: P0 — FOUNDATION FOR ALL DOWNSTREAM AI PIPELINES

---

1. ARCHITECTURAL PURPOSE

Phase 4A is the **data ingestion and structural refinement layer**. It takes raw HTML from the Scrapy pipeline (Phases 1-3) and transforms it into clean, structured, multi-format outputs that serve three downstream consumers:

Consumer	Format	Use Case
Human analysts	Markdown + CSV/JSON	Readable reports, quick inspection
ML pipelines	Parquet	Columnar, compressed, schema-enforced
Phase 4B RAG	Markdown chunks + metadata	LLM context windows, semantic search

Core principle: One crawl → multiple formats → one unified schema. No data is lost; every field is traceable back to its source.

---

2. DATA FLOW

```
[Raw HTML from Scrapy Pipeline]
            |
            v
+-------------------------------------+
|  Step 1: MarkdownExtractionPipeline |  Priority 110
|  (Trafilatura: HTML → clean MD)     |
+-------------+-----------------------+
              |
              v
+-------------------------------------+
|  Step 2: MultimodalAssetExtractor   |  Inline (called by pipeline)
|  (Images, videos, alt-text isolation)|
+-------------+-----------------------+
              |
              v
+-------------------------------------+
|  Step 3: UnifiedSchemaEnricher      |  Inline
|  (Schema validation + defaults)     |
```

```
+-------------+----------------------+
              |
    +---------+-----------+
    |                     |
    v                     v
+----------+    +------------------+
| Parquet  |    | JSON / CSV / MD  |  (existing export pipeline)
| Export   |    | Per-page files   |
| (NEW)    |    | (EXISTING)       |
+----+-----+    +------------------+
     |
     v
+-------------------------------------+
|  Step 4: MetadataStore (SQLite)     |  Priority 160 (NEW)
|  (Unified relational schema)        |
+-------------------------------------+
```

---

3. COMPONENT SPECIFICATIONS

3.1 MarkdownExtractionPipeline

File: `nexora_crawler/pipelines/markdown_pipeline.py`
Priority: 110 (after ExtractionPipeline at 100, before StylePipeline at 150)
Purpose: Convert raw HTML to clean, LLM-ready Markdown.

3.1.1 Implementation

```python
# markdown_pipeline.py
# MarkdownExtractionPipeline — Phase 4A Core
# Converts raw HTML to clean Markdown using Trafilatura.
# Priority: 110

import logging
import trafilatura

logger = logging.getLogger(__name__)


class MarkdownExtractionPipeline:
    """
    Scrapy pipeline converting HTML → clean Markdown.
    Uses Trafilatura for intelligent boilerplate removal.
    Preserves tables, links; strips nav, footers, ads.
    """

    def __init__(self):
        self.stats = {
            "pages_processed": 0,
            "markdown_generated": 0,
            "extraction_failures": 0,
```

```
            "avg_token_reduction": 0.0,
        }

    @classmethod
    def from_crawler(cls, crawler):
        return cls()

    async def process_item(self, item, spider):
        html = item.get("html", "")
        if not html:
            item["markdown"] = ""
            item["extraction_method"] = "no_html"
            return item

        try:
            markdown = trafilatura.extract(
                html,
                output_format="markdown",
                include_comments=False,
                include_tables=True,
                include_images=False,      # Images handled by MultimodalAssetExtractor
                include_links=True,
                deduplicate=True,
                url=item.get("url", ""),
            )

            if markdown and len(markdown.strip()) > 50:
                item["markdown"] = markdown.strip()
                item["markdown_word_count"] = len(markdown.split())
                item["extraction_method"] = "trafilatura"

                # Token reduction metric
                raw_tokens = len(html) / 4
                clean_tokens = len(markdown) / 4
                if raw_tokens > 0:
                    item["token_reduction_pct"] = round(
                        (1 - clean_tokens / raw_tokens) * 100, 1
                    )

                self.stats["markdown_generated"] += 1
            else:
                # Fallback: use clean_text if Trafilatura fails
                clean_text = item.get("clean_text", "")
                item["markdown"] = clean_text
                item["markdown_word_count"] = len(clean_text.split()) if clean_text else 0
                item["extraction_method"] = "trafilatura_fallback_to_clean_text"
                item["token_reduction_pct"] = 0.0

            self.stats["pages_processed"] += 1

        except Exception as exc:
```

```
            logger.error("[MarkdownPipeline] Extraction failed for %s: %s",
                         item.get("url", "unknown"), exc)
            item["markdown"] = item.get("clean_text", "")
            item["extraction_method"] = "error_fallback"
            item["token_reduction_pct"] = 0.0
            self.stats["extraction_failures"] += 1

        return item

    def close_spider(self, spider):
        logger.info("[MarkdownPipeline] Stats: %s", self.stats)
```

3.1.2 Configuration (settings.py)

```
# Markdown Pipeline Settings
NEXORA_MARKDOWN_ENABLED = True
NEXORA_MARKDOWN_INCLUDE_COMMENTS = False
NEXORA_MARKDOWN_INCLUDE_TABLES = True
NEXORA_MARKDOWN_INCLUDE_LINKS = True
NEXORA_MARKDOWN_DEDUPLICATE = True

# Pipeline registration
ITEM_PIPELINES = {
    'nexora_crawler.pipelines.NexoraExtractionPipeline': 100,
    'nexora_crawler.pipelines.markdown_pipeline.MarkdownExtractionPipeline': 110,
    'nexora_crawler.pipelines.NexoraStylePipeline': 150,
    # ... Phase 4B at 250, Parquet at 450, etc.
}
```

3.1.3 Item Fields Added

Field	Type	Description
`markdown`	`str`	Clean Markdown content
`markdown_word_count`	`int`	Word count of Markdown
`extraction_method`	`str`	`trafilatura` / `fallback` / `error`
`token_reduction_pct`	`float`	% of tokens reduced vs raw HTML

3.2 MultimodalAssetExtractor

File: `nexora_crawler/extractors/multimodal_extractor.py`
Called by: MarkdownExtractionPipeline (inline, not a separate Scrapy pipeline)
Purpose: Isolate images, videos, and their metadata from HTML.

3.2.1 Implementation

```
# multimodal_extractor.py
# MultimodalAssetExtractor — Phase 4A
```

```python
# Isolates images, videos, and alt-text from HTML.
# Produces structured asset metadata without downloading binaries.

import logging
from typing import List, Dict
from urllib.parse import urljoin

from bs4 import BeautifulSoup

logger = logging.getLogger(__name__)


class MultimodalAssetExtractor:
    """
    Extracts image and video references from HTML.
    Does NOT download binaries — only captures metadata and URLs.
    """

    def extract(self, html: str, base_url: str = "") -> Dict:
        """
        Returns structured asset metadata.

        Returns:
            {
                "images": [
                    {
                        "src": "https://.../image.jpg",
                        "alt": "Description",
                        "width": "800",
                        "height": "600",
                        "loading": "lazy",
                        "is_hero": False,
                    }
                ],
                "videos": [
                    {
                        "src": "https://.../video.mp4",
                        "poster": "...",
                        "type": "video/mp4",
                    }
                ],
                "total_images": 5,
                "total_videos": 1,
                "has_hero_image": False,
            }
        """
        if not html:
            return self._empty_result()

        soup = BeautifulSoup(html, "lxml")
```

```
        images = self._extract_images(soup, base_url)
        videos = self._extract_videos(soup, base_url)

        # Hero image heuristic: first large image above the fold
        has_hero = False
        if images:
            first_img = images[0]
            width = int(first_img.get("width") or 0)
            height = int(first_img.get("height") or 0)
            if width >= 600 or height >= 400:
                first_img["is_hero"] = True
                has_hero = True

        return {
            "images": images,
            "videos": videos,
            "total_images": len(images),
            "total_videos": len(videos),
            "has_hero_image": has_hero,
        }

    def _extract_images(self, soup: BeautifulSoup, base_url: str) -> List[Dict]:
        images = []
        for img in soup.find_all("img"):
            src = img.get("src", "")
            if src:
                src = urljoin(base_url, src)

            # Also check srcset for highest resolution
            srcset = img.get("srcset", "")
            best_src = src
            if srcset:
                candidates = [
                    (urljoin(base_url, part.strip().split()[0]),
                     int(part.strip().split()[1].replace("w", ""))
                     if len(part.strip().split()) > 1 else 0)
                    for part in srcset.split(",")
                ]
                if candidates:
                    best_src = max(candidates, key=lambda x: x[1])[0]

            images.append({
                "src": best_src,
                "alt": img.get("alt", ""),
                "width": img.get("width", ""),
                "height": img.get("height", ""),
                "loading": img.get("loading", "eager"),
                "is_hero": False,
            })
        return images
```

```
    def _extract_videos(self, soup: BeautifulSoup, base_url: str) -> List[Dict]:
        videos = []

        # <video> tags
        for vid in soup.find_all("video"):
            src = vid.get("src", "")
            if not src:
                source = vid.find("source")
                src = source.get("src", "") if source else ""

            if src:
                videos.append({
                    "src": urljoin(base_url, src),
                    "poster": urljoin(base_url, vid.get("poster", "")),
                    "type": vid.get("type", ""),
                    "width": vid.get("width", ""),
                    "height": vid.get("height", ""),
                })

        # iframe embeds (YouTube, Vimeo, etc.)
        for iframe in soup.find_all("iframe"):
            src = iframe.get("src", "")
            if any(domain in src for domain in ["youtube", "vimeo", "dailymotion"]):
                videos.append({
                    "src": src,
                    "poster": "",
                    "type": "embed",
                    "platform": "youtube" if "youtube" in src else "vimeo",
                })

        return videos

    def _empty_result(self) -> Dict:
        return {
            "images": [],
            "videos": [],
            "total_images": 0,
            "total_videos": 0,
            "has_hero_image": False,
        }
```

3.2.2 Integration into MarkdownExtractionPipeline Add to `process_item` in MarkdownExtractionPipeline, after markdown extraction:

```
from nexora_crawler.extractors.multimodal_extractor import MultimodalAssetExtractor

asset_extractor = MultimodalAssetExtractor()
assets = asset_extractor.extract(html, item.get("url", ""))
item["image_assets"] = assets["images"]
item["video_assets"] = assets["videos"]
item["total_images"] = assets["total_images"]
```

```
item["total_videos"] = assets["total_videos"]
item["has_hero_image"] = assets["has_hero_image"]
```

3.3 UnifiedSchemaEnricher

File: `nexora_crawler/pipelines/schema_enricher.py`
Priority: 160 (after StylePipeline at 150, before Phase 4B at 250)
Purpose: Ensure every item conforms to the unified schema with defaults.

3.3.1 Unified Schema Definition Every record produced by Phase 4A MUST contain these fields:

```python
# models.py — Unified Schema Dataclass
# Every record must conform. Missing fields populated with defaults.

from typing import List, Dict, Any, Optional
from dataclasses import dataclass, field


@dataclass
class NexoraUnifiedRecord:
    # — Identity —
    url: str = ""
    title: str = ""
    domain: str = ""

    # — Temporal —
    timestamp: str = ""           # ISO 8601 UTC
    crawl_id: str = ""          # UUID of crawl job

    # — Content —
    markdown_content: str = ""    # Clean Markdown (primary)
    clean_text: str = ""        # Fallback plain text
    html_length: int = 0
    markdown_word_count: int = 0
    token_reduction_pct: float = 0.0

    # — AI Enrichment (populated by Phase 4B) —
    ai_summary: str = ""        # 2-3 sentence summary
    ai_tags: List[str] = field(default_factory=list)
    ai_embedding: List[float] = field(default_factory=list)

    # — Entities —
    entities: Dict[str, Any] = field(default_factory=lambda: {
        "prices": [],
        "currency": "",
        "tickers": [],
        "products": [],
        "people": [],
        "organizations": [],
```

```
    })
    price_change_delta: Optional[float] = None

    # — Style Analysis —
    style_analysis: Dict[str, Any] = field(default_factory=lambda: {
        "dominant_colors": [],
        "tech_stack": [],
        "css_framework": "",
        "theme": "",
        "fonts": [],
    })

    # — Quality Scores —
    quality_scores: Dict[str, float] = field(default_factory=lambda: {
        "readability": 0.0,
        "duplication_score": 0.0,
        "text_density": 0.0,
        "crawl_quality": 1.0,
    })

    # — Multimodal —
    image_assets: List[Dict] = field(default_factory=list)
    video_assets: List[Dict] = field(default_factory=list)
    total_images: int = 0
    total_videos: int = 0
    has_hero_image: bool = False

    # — Metadata —
    language: str = ""
    website_type: str = "unknown"  # article, product, docs, blog, etc.
    extraction_method: str = ""

    # — Provenance —
    spider_name: str = ""
    depth: int = 0
    playwright_used: bool = False

    def to_dict(self) -> Dict[str, Any]:
        from dataclasses import asdict
        return asdict(self)

    def to_parquet_row(self) -> Dict[str, Any]:
        import json
        d = self.to_dict()
        # Flatten nested dicts to JSON strings for Parquet
        d["entities_json"] = json.dumps(d.pop("entities"))
        d["style_analysis_json"] = json.dumps(d.pop("style_analysis"))
        d["quality_scores_json"] = json.dumps(d.pop("quality_scores"))
        d["image_assets_json"] = json.dumps(d.pop("image_assets"))
        d["video_assets_json"] = json.dumps(d.pop("video_assets"))
        d["ai_tags_json"] = json.dumps(d.pop("ai_tags"))
```

```
        d["ai_embedding_json"] = json.dumps(d.pop("ai_embedding"))
        # Remove heavy text fields (stored separately)
        d.pop("markdown_content", None)
        d.pop("clean_text", None)
        return d
```

3.3.2 Enricher Implementation

```
# schema_enricher.py
# UnifiedSchemaEnricher — Phase 4A
# Ensures every item conforms to the unified schema.
# Runs after StylePipeline (150), before Phase 4B (250).
# Priority: 160

import logging
from datetime import datetime, timezone

logger = logging.getLogger(__name__)


class UnifiedSchemaEnricher:
    """
    Scrapy pipeline that enforces the unified schema.
    Populates defaults, validates types, adds temporal fields.
    """

    def __init__(self):
        self.stats = {"items_enriched": 0, "defaults_applied": 0}

    @classmethod
    def from_crawler(cls, crawler):
        return cls()

    async def process_item(self, item, spider):
        # Ensure crawl_id
        if not item.get("crawl_id"):
            item["crawl_id"] = getattr(spider, "crawl_id", "")

        # Ensure timestamp
        if not item.get("timestamp"):
            item["timestamp"] = datetime.now(timezone.utc).isoformat()

        # Ensure domain
        if not item.get("domain"):
            url = item.get("url", "")
            item["domain"] = url.split("/")[2] if "//" in url else ""

        # Ensure entities with defaults
        if not item.get("entities"):
            item["entities"] = {
                "prices": [],
```

```
                "currency": "",
                "tickers": [],
                "products": [],
                "people": [],
                "organizations": [],
            }
            self.stats["defaults_applied"] += 1

        # Ensure style_analysis with defaults
        if not item.get("style_analysis"):
            item["style_analysis"] = {
                "dominant_colors": [],
                "tech_stack": [],
                "css_framework": "",
                "theme": "",
                "fonts": [],
            }

        # Ensure quality_scores with defaults
        if not item.get("quality_scores"):
            item["quality_scores"] = {
                "readability": 0.0,
                "duplication_score": 0.0,
                "text_density": 0.0,
                "crawl_quality": 1.0,
            }

        # Ensure website_type classification
        if not item.get("website_type"):
            item["website_type"] = self._classify_website_type(item)

        self.stats["items_enriched"] += 1
        return item

    def _classify_website_type(self, item) -> str:
        """Heuristic classification of page type."""
        url = item.get("url", "").lower()
        markdown = item.get("markdown", "")
        title = item.get("title", "").lower()

        if any(x in url for x in ["/product", "/item", "/shop", "/store", "/cart"]):
            return "e-commerce"
        if any(x in title for x in ["blog", "article", "post", "news"]):
            return "blog"
        if any(x in url for x in ["/docs", "/documentation", "/api", "/guide"]):
            return "documentation"
        if item.get("entities", {}).get("prices"):
            return "e-commerce"
        if len(markdown) > 2000 and "##" in markdown:
            return "article"
        return "unknown"
```

```
    def close_spider(self, spider):
        logger.info("[SchemaEnricher] Stats: %s", self.stats)
```

3.4 Multi-Format Export Compiler (Parquet)

File: `nexora_crawler/pipelines/parquet_export.py`
Priority: 450 (after Phase 4B at 250, before standard export at 500)
Purpose: Export data as compressed Apache Parquet files.

3.4.1 Implementation

```
# parquet_export.py
# ParquetExportPipeline — Phase 4A Analytical Storage
# Exports crawled data as compressed Apache Parquet files.
# Priority: 450

import json
import logging
import os
from datetime import datetime, timezone

import pandas as pd
import pyarrow as pa
import pyarrow.parquet as pq

logger = logging.getLogger(__name__)


class ParquetExportPipeline:
    """
    Scrapy pipeline exporting data as Apache Parquet.
    Buffers rows and flushes to disk in batches.
    """

    def __init__(self, crawler):
        self.crawler = crawler
        self.settings = crawler.settings
        self.enabled = self.settings.getbool('NEXORA_PARQUET_ENABLED', True)
        self.compression = self.settings.get('NEXORA_PARQUET_COMPRESSION', 'snappy')
        self.row_group_size = self.settings.getint('NEXORA_PARQUET_ROW_GROUP_SIZE', 10000)
        self.output_dir = self.settings.get('NEXORA_PARQUET_OUTPUT', './output/parquet')

        self._buffer = []
        self._buffer_size = 100
        self._total_rows = 0
        self._file_counter = 0

    @classmethod
```

```python
    def from_crawler(cls, crawler):
        return cls(crawler)

    def open_spider(self, spider):
        if not self.enabled:
            return
        os.makedirs(self.output_dir, exist_ok=True)
        logger.info("[Parquet] Export enabled — dir: %s", self.output_dir)

    async def process_item(self, item, spider):
        if not self.enabled:
            return item

        row = self._item_to_parquet_row(item)
        self._buffer.append(row)

        if len(self._buffer) >= self._buffer_size:
            self._flush_buffer(spider)

        return item

    def close_spider(self, spider):
        if not self.enabled:
            return
        if self._buffer:
            self._flush_buffer(spider)
        logger.info("[Parquet] Total rows exported: %d", self._total_rows)

    def _item_to_parquet_row(self, item: dict) -> dict:
        row = dict(item)

        # Serialize nested structures to JSON strings
        for key in ['entities', 'style_analysis', 'quality_scores',
                    'image_assets', 'video_assets', 'ai_tags', 'ai_embedding']:
            if key in row and not isinstance(row[key], str):
                row[f"{key}_json"] = json.dumps(row[key])
                del row[key]

        # Remove heavy text fields (stored separately in Markdown/JSON exports)
        for key in ['html', 'markdown', 'clean_text']:
            row.pop(key, None)

        return row

    def _flush_buffer(self, spider):
        if not self._buffer:
            return

        try:
            df = pd.DataFrame(self._buffer)
            timestamp = datetime.now(timezone.utc).strftime('%Y%m%d_%H%M%S')
```

```
            filename = f"{spider.name}_{timestamp}_{self._file_counter:04d}.parquet"
            filepath = os.path.join(self.output_dir, filename)

            table = pa.Table.from_pandas(df)
            pq.write_table(
                table,
                filepath,
                compression=self.compression,
                row_group_size=self.row_group_size,
                use_dictionary=True,
                write_statistics=True,
            )

            self._total_rows += len(self._buffer)
            self._file_counter += 1
            logger.info("[Parquet] Wrote %d rows to %s", len(self._buffer), filename)
            self._buffer = []

        except Exception as exc:
            logger.error("[Parquet] Flush failed: %s", exc)
```

3.4.2 Configuration

```
# settings.py
NEXORA_PARQUET_ENABLED = True
NEXORA_PARQUET_COMPRESSION = 'snappy'  # snappy | gzip | brotli | zstd
NEXORA_PARQUET_ROW_GROUP_SIZE = 10000
NEXORA_PARQUET_OUTPUT = './output/parquet'
```

3.5 MetadataStore (SQLite)

File: `nexora_crawler/storage/local_sqlite.py`
Purpose: Relational metadata storage for fast filtering and analytics.

3.5.1 Implementation

```
# local_sqlite.py
# MetadataStore — Phase 4A Unified Relational Storage
# SQLite-backed metadata for fast filtering and analytics.
# Uses the unified schema. Replaces old Phase 3B metadata_store.py.

import os
import json
import logging
import sqlite3
from typing import List, Dict

logger = logging.getLogger(__name__)
```

```
class MetadataStore:
    """
    SQLite metadata store with unified schema.
    Tables: pages, crawl_jobs
    """

    def __init__(self, db_path: str = "./data/nexora_metadata.db"):
        self.db_path = db_path
        os.makedirs(os.path.dirname(db_path) or ".", exist_ok=True)
        self._init_schema()

    def _init_schema(self):
        with sqlite3.connect(self.db_path) as conn:
            conn.executescript("""
                CREATE TABLE IF NOT EXISTS pages (
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    url TEXT NOT NULL UNIQUE,
                    domain TEXT NOT NULL,
                    title TEXT,
                    timestamp TEXT NOT NULL,
                    crawl_id TEXT NOT NULL,
                    markdown_preview TEXT,
                    markdown_word_count INTEGER DEFAULT 0,
                    token_reduction_pct REAL DEFAULT 0.0,
                    ai_summary TEXT,
                    ai_tags_json TEXT,
                    entities_json TEXT DEFAULT '{}',
                    price_change_delta REAL,
                    style_analysis_json TEXT DEFAULT '{}',
                    quality_scores_json TEXT DEFAULT '{}',
                    image_assets_json TEXT DEFAULT '[]',
                    video_assets_json TEXT DEFAULT '[]',
                    total_images INTEGER DEFAULT 0,
                    total_videos INTEGER DEFAULT 0,
                    has_hero_image INTEGER DEFAULT 0,
                    language TEXT,
                    website_type TEXT DEFAULT 'unknown',
                    extraction_method TEXT,
                    spider_name TEXT,
                    depth INTEGER DEFAULT 0,
                    playwright_used INTEGER DEFAULT 0,
                    created_at TEXT DEFAULT (datetime('now'))
                );

                CREATE INDEX IF NOT EXISTS idx_pages_domain ON pages(domain);
                CREATE INDEX IF NOT EXISTS idx_pages_crawl_id ON pages(crawl_id);
                CREATE INDEX IF NOT EXISTS idx_pages_website_type ON pages(website_type);
                CREATE INDEX IF NOT EXISTS idx_pages_timestamp ON pages(timestamp);
                CREATE INDEX IF NOT EXISTS idx_pages_language ON pages(language);
```

```
            CREATE TABLE IF NOT EXISTS crawl_jobs (
                job_id TEXT PRIMARY KEY,
                url TEXT NOT NULL,
                strategy TEXT DEFAULT 'whole-website',
                max_pages INTEGER DEFAULT 100,
                status TEXT DEFAULT 'running',
                pages_crawled INTEGER DEFAULT 0,
                pages_failed INTEGER DEFAULT 0,
                started_at TEXT NOT NULL,
                completed_at TEXT,
                error TEXT
            );
        """)
        conn.commit()
    logger.info("[MetadataStore] Schema initialized at %s", self.db_path)

def insert_page(self, item: dict) -> bool:
    try:
        with sqlite3.connect(self.db_path) as conn:
            conn.execute("""
                INSERT OR REPLACE INTO pages (
                    url, domain, title, timestamp, crawl_id,
                    markdown_preview, markdown_word_count, token_reduction_pct,
                    ai_summary, ai_tags_json, entities_json, price_change_delta,
                    style_analysis_json, quality_scores_json,
                    image_assets_json, video_assets_json,
                    total_images, total_videos, has_hero_image,
                    language, website_type, extraction_method,
                    spider_name, depth, playwright_used
                ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
            """, (
                item.get("url", ""),
                item.get("domain", ""),
                item.get("title", ""),
                item.get("timestamp", ""),
                item.get("crawl_id", ""),
                item.get("markdown", "")[:500],
                item.get("markdown_word_count", 0),
                item.get("token_reduction_pct", 0.0),
                item.get("ai_summary", ""),
                json.dumps(item.get("ai_tags", [])),
                json.dumps(item.get("entities", {})),
                item.get("price_change_delta"),
                json.dumps(item.get("style_analysis", {})),
                json.dumps(item.get("quality_scores", {})),
                json.dumps(item.get("image_assets", [])),
                json.dumps(item.get("video_assets", [])),
                item.get("total_images", 0),
                item.get("total_videos", 0),
                1 if item.get("has_hero_image") else 0,
                item.get("language", ""),
```

```
                    item.get("website_type", "unknown"),
                    item.get("extraction_method", ""),
                    item.get("spider_name", ""),
                    item.get("depth", 0),
                    1 if item.get("playwright_used") else 0,
                ))
                conn.commit()
            return True
        except Exception as exc:
            logger.error("[MetadataStore] Insert failed for %s: %s",
                         item.get("url", ""), exc)
            return False

    def query_by_domain(self, domain: str, limit: int = 100) -> List[Dict]:
        with sqlite3.connect(self.db_path) as conn:
            conn.row_factory = sqlite3.Row
            cursor = conn.execute(
                "SELECT * FROM pages WHERE domain = ? ORDER BY timestamp DESC LIMIT ?",
                (domain, limit)
            )
            return [dict(row) for row in cursor.fetchall()]

    def query_by_crawl_id(self, crawl_id: str) -> List[Dict]:
        with sqlite3.connect(self.db_path) as conn:
            conn.row_factory = sqlite3.Row
            cursor = conn.execute(
                "SELECT * FROM pages WHERE crawl_id = ? ORDER BY timestamp DESC",
                (crawl_id,)
            )
            return [dict(row) for row in cursor.fetchall()]

    def get_stats(self) -> Dict:
        with sqlite3.connect(self.db_path) as conn:
            total = conn.execute("SELECT COUNT(*) FROM pages").fetchone()[0]
            domains = conn.execute(
                "SELECT COUNT(DISTINCT domain) FROM pages"
            ).fetchone()[0]
            return {"total_pages": total, "unique_domains": domains}
```

3.5.2 Pipeline Integration

```
# metadata_indexer.py
# MetadataIndexerPipeline — Phase 4A
# Scrapy pipeline that indexes items into MetadataStore.
# Priority: 165 (after UnifiedSchemaEnricher at 160)

import logging
from nexora_crawler.storage.local_sqlite import MetadataStore

logger = logging.getLogger(__name__)
```

```
class MetadataIndexerPipeline:
    def __init__(self, crawler):
        self.store = MetadataStore(
            db_path=crawler.settings.get('NEXORA_METADATA_DB', './data/nexora_metadata.db')
        )
        self.stats = {"indexed": 0, "failed": 0}

    @classmethod
    def from_crawler(cls, crawler):
        return cls(crawler)

    async def process_item(self, item, spider):
        success = self.store.insert_page(dict(item))
        if success:
            self.stats["indexed"] += 1
        else:
            self.stats["failed"] += 1
        return item

    def close_spider(self, spider):
        logger.info("[MetadataIndexer] Stats: %s", self.stats)
        stats = self.store.get_stats()
        logger.info("[MetadataStore] DB stats: %s", stats)
```

---

4. ITEMS.PY UPDATE

Add these fields to `NexoraPageItem`:

```
import scrapy


class NexoraPageItem(scrapy.Item):
    # — Phase 1-3 Existing Fields —
    url = scrapy.Field()
    status = scrapy.Field()
    html = scrapy.Field()
    depth = scrapy.Field()
    spider_name = scrapy.Field()
    crawled_at = scrapy.Field()
    playwright_used = scrapy.Field()

    title = scrapy.Field()
    description = scrapy.Field()
    headings = scrapy.Field()
    images = scrapy.Field()
    internal_links = scrapy.Field()
    clean_text = scrapy.Field()
    word_count_raw = scrapy.Field()
```

```
    word_count_clean = scrapy.Field()
    fingerprint = scrapy.Field()
    language_iso = scrapy.Field()
    structured_schema = scrapy.Field()
    social_graphs = scrapy.Field()
    graph_relations = scrapy.Field()
    image_assets = scrapy.Field()
    styles = scrapy.Field()

    saved_json = scrapy.Field()
    saved_csv = scrapy.Field()
    __skip = scrapy.Field()

    # — Phase 4A: Markdown & Content —
    markdown = scrapy.Field()
    markdown_word_count = scrapy.Field()
    extraction_method = scrapy.Field()
    token_reduction_pct = scrapy.Field()

    # — Phase 4A: Multimodal —
    image_assets = scrapy.Field()      # list[dict] — structured image metadata
    video_assets = scrapy.Field()        # list[dict] — structured video metadata
    total_images = scrapy.Field()
    total_videos = scrapy.Field()
    has_hero_image = scrapy.Field()

    # — Phase 4A: Unified Schema —
    crawl_id = scrapy.Field()
    timestamp = scrapy.Field()
    domain = scrapy.Field()
    entities = scrapy.Field()
    price_change_delta = scrapy.Field()
    style_analysis = scrapy.Field()
    quality_scores = scrapy.Field()
    website_type = scrapy.Field()

    # — Phase 4B: AI Enrichment (populated later) —
    ai_summary = scrapy.Field()
    ai_tags = scrapy.Field()
    ai_embedding = scrapy.Field()

    # — Phase 4B: Chunking (populated later) —
    chunk_count = scrapy.Field()
    chunk_ids = scrapy.Field()
    has_embedding = scrapy.Field()
```

---

5. SETTINGS.PY UPDATE

```
# — Phase 4A: Markdown Pipeline —
```

```
NEXORA_MARKDOWN_ENABLED = True

# — Phase 4A: Parquet Export —
NEXORA_PARQUET_ENABLED = True
NEXORA_PARQUET_COMPRESSION = 'snappy'
NEXORA_PARQUET_ROW_GROUP_SIZE = 10000
NEXORA_PARQUET_OUTPUT = './output/parquet'

# — Phase 4A: Metadata Store —
NEXORA_METADATA_DB = './data/nexora_metadata.db'

# — Pipeline Registration (Complete) —
ITEM_PIPELINES = {
    'nexora_crawler.pipelines.NexoraExtractionPipeline': 100,
    'nexora_crawler.pipelines.markdown_pipeline.MarkdownExtractionPipeline': 110,
    'nexora_crawler.pipelines.NexoraStylePipeline': 150,
    'nexora_crawler.pipelines.schema_enricher.UnifiedSchemaEnricher': 160,
    'nexora_crawler.pipelines.metadata_indexer.MetadataIndexerPipeline': 165,
    # Phase 4B pipelines at 250+
    'nexora_crawler.pipelines.parquet_export.ParquetExportPipeline': 450,
    'nexora_crawler.pipelines.NexoraExportPipeline': 500,
    'nexora_crawler.pipelines.NexoraDatasetPipeline': 600,
}
```

---

6. TEST MATRIX

Test ID	Scenario	Expected Result
P4A-T01	Trafilatura extracts Markdown from HTML	`markdown` field populated, `token_reduction_pct` > 50%
P4A-T02	Boilerplate removal	No “cookie policy”, “subscribe”, “navigation” in markdown
P4A-T03	Table preservation	HTML `<table>` → Markdown pipe-delimited table
P4A-T04	Image asset extraction	`image_assets` contains src, alt, width, height
P4A-T05	Video asset extraction	`video_assets` contains src, poster, platform
P4A-T06	Unified schema defaults	All records have `entities`, `style_analysis`, `quality_scores`
P4A-T07	Website type classification	`website_type` correctly identifies blog, product, docs
P4A-T08	Parquet export	`.parquet` file created, readable by pandas
P4A-T09	Parquet compression	File size < 30% of equivalent JSON
P4A-T10	Metadata store insert	Record queryable by domain, crawl_id
P4A-T11	Schema enrichment	Missing fields populated with defaults, not omitted
P4A-T12	No regression	Phase 3 tests still pass

---

7. DEFINITION OF DONE

- `MarkdownExtractionPipeline` converts HTML → clean Markdown
- `MultimodalAssetExtractor` isolates images/videos with metadata
- `UnifiedSchemaEnricher` enforces schema with defaults for all fields
- `ParquetExportPipeline` writes compressed Parquet files
- `MetadataIndexerPipeline` stores records in SQLite with unified schema
- `items.py` updated with all Phase 4A fields
- `settings.py` updated with pipeline priorities
- All 12 test cases pass
- Phase 3 tests show no regression
- Parquet files readable by `pd.read_parquet()`
- Metadata store queries return correct results by domain and crawl_id